# EXPERIMENT No. 8

**Date of Performance:**

**Date of Submission:**

**Aim:** To design and implement load balancing in a distributed system in order to efficiently distribute workload among multiple processes or nodes, ensuring optimal resource utilization, improved system performance, reduced response time, and prevention of overload on any single node.

## OBJECTIVES:

1. To study the concept of load balancing and its importance in distributed systems for efficient resource utilization.
2. To understand different load estimation policies and threshold-based decision making.
3. To analyze various process transfer policies and their impact on system performance.
4. To implement state information exchange mechanisms for load monitoring across nodes.
5. To evaluate different location policies (threshold method vs. shortest method) for process migration.
6. To observe how priority assignment and migration limiting policies affect overall system stability.

## SCOPE OF STUDY:

**Resource Utilization and Load Estimation**

This scope focuses on studying the concept of load balancing in distributed systems and understanding its importance in achieving efficient resource utilization. It also covers different load estimation policies and threshold-based decision-making techniques to determine when a system is overloaded or underloaded.

**Process Migration and Performance Analysis**

This scope involves analyzing various process transfer policies and evaluating their impact on overall system performance. It further includes the study of different location policies such as threshold-based and shortest-load methods to determine the most suitable node for process migration.

**Load Monitoring and System Stability**

This scope emphasizes the implementation of state information exchange mechanisms for monitoring load across distributed nodes. It also examines how priority assignment and migration limiting policies contribute to maintaining system stability and preventing excessive process movement.

## 1. Literature Survery:-

**Theory:**

**Load Balancing in Distributed Systems:**

Load balancing is the way of distributing load units (jobs or tasks) across a set of processors which are connected to a network that may be distributed across the globe. The primary goal is to ensure that no single processor is overwhelmed while others remain idle, thereby optimizing resource utilization and minimizing response time.

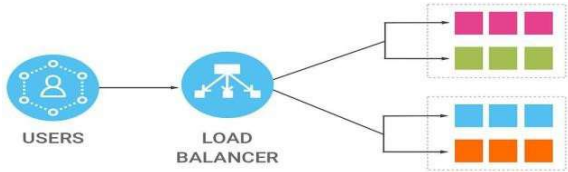
- The excess load or remaining unexecuted load from a processor is migrated to other processors which have load below the threshold load.

- Threshold load is such an amount of load on a processor that any additional load may cause performance degradation.
- By load balancing strategy, it is possible to make every processor equally busy and to finish the work approximately at the same time.

**Load Balancing Approaches:**

Load balancing algorithms can be broadly classified into:

1. **Static Load Balancing**: Decisions are made at compile time based on prior knowledge of system behavior.
2. **Dynamic Load Balancing**: Decisions are made at runtime based on current system state information.




**Issues in Designing Load-balancing Algorithms:**

1. **Load Estimation Policy:**

This determines how to estimate the workload of a node. Some measurable parameters (with time and node dependent factor) can be:

- Total number of processes on the node
- Resource demands of these processes
- Instruction mixes of these processes
- Architecture and speed of the node's processor

Several load-balancing algorithms use the total number of processes as a simple yet effective metric.

2. **Process Transfer Policy:**

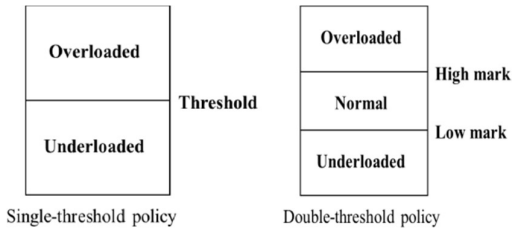
This determines whether to execute a process locally or remotely. Most algorithms use the **threshold policy** to decide whether a node is lightly-loaded or heavily-loaded.

- **Threshold value** is a limiting value of the workload of node which can be determined by:

- **Static policy**: Predefined threshold value for each node depending on processing capability

- **Dynamic policy**: Threshold value is calculated from average workload and a predefined constant
- Below threshold value, node accepts processes to execute; above threshold value, node tries to transfer processes to a lightly-loaded node.

- Single-threshold policy may lead to unstable algorithm because an underloaded node could become overloaded right after a process migration




**3. State Information Exchange Policy:**

This determines how to exchange load information among nodes. Dynamic policies require frequent exchange of state information, but these extra messages have two opposite impacts:

- Increasing the number of messages gives more accurate scheduling decisions
- Increasing the number of messages raises the queuing time of messages

State information policies can be:

- **Periodic broadcast**: Nodes broadcast their load state at regular intervals
- **Broadcast when state changes**: Only send updates when load crosses thresholds
- **On-demand exchange**: Request information only when needed
- **Exchange by polling**: Actively query other nodes for their load status

**4. Location Policy:**

This determines to which node the transferable process should be sent.

**a) Threshold Method:** Policy selects a random node, checks whether the node is able to receive the process, then transfers the process. If node rejects, another node is selected randomly. This continues until the probe limit is reached.

**b) Shortest Method:** L distinct nodes are chosen at random, each is polled to determine its load. The process is transferred to the node having the minimum value unless its workload value prohibits accepting the process. A simple improvement is to discontinue probing whenever a node with zero load is encountered.

**5. Priority Assignment Policy:**

This determines the priority of execution of local and remote processes.

- **Selfish**: Local processes are given higher priority than remote processes. This provides worst response time performance among the three policies.
- **Altruistic**: Remote processes are given higher priority than local processes. This provides best response time performance among the three policies.
- **Intermediate**: When the number of local processes is greater than or equal to the number of remote processes, local processes are given higher priority. Otherwise, remote processes are given higher priority.

**6. Migration Limiting Policy:**

This policy determines the total number of times a process can migrate.

- **Uncontrolled**: A remote process arriving at a node is treated just as a process originating at a node, so a process may be migrated any number of times.
- **Controlled**: Avoids the instability of the uncontrolled policy by using a migration count parameter to fix a limit on the number of times a process can migrate.
- **Irrevocable migration policy**: Migration count is fixed to 1.
- For long execution processes, migration count must be greater than 1 to adapt for dynamically changing states.

**2.1 Algorithm:-**

**Algorithm: Load Balancing in Distributed System**

**Input:**

Set of nodes: $N = \{N_1, N_2, \ldots, N_n\}$
Load on each node: $L = \{L_1, L_2, \ldots, L_n\}$
Threshold value: $T$
Set of processes: $P = \{P_1, P_2, \ldots, P_m\}$
**Output:**

- Balanced distribution of processes across nodes
- Reduced load difference among all nodes

**Steps:**

1. Initialize all nodes $N_i$with their respective loads $L_i$.
2. For each node $N_i$, compute the current load $L_i$.
3. Compare each node's load with threshold $T$:
   If $L_i > T$, mark node as Overloaded.
   If $L_i < T$, mark node as Underloaded.
4. Select the source node $N_s$such that:
$$N_s = \arg\max (L_i)$$
5. Select the destination node $N_d$such that:
$$N_d = \arg\min (L_i)$$
6. Select a process $P_k$from the overloaded node $N_s$.
7. Check migration condition:
   If $(L_s - L_d) > T$, proceed to migration.
   Otherwise, terminate the process.
8. Transfer process $P_k$from source node $N_s$to destination node $N_d$.
9. Update the loads of nodes:
$$L_s = L_s - w_k$$
$$L_d = L_d + w_k$$

where $w_k$is the workload of process $P_k$.
10. Repeat steps 3 to 9 until the load difference satisfies:
$$| L_i - L_j | \leq T \forall i, j$$
11. Stop when all nodes are balanced and output the final state.

**2.2 Programming Language Used**

**2.2.1.1 Libraries Used**

**1. Input-Output Library (<iostream>)**

**Purpose:**
Used for displaying output on the console.

**Syntax:**
```
#include <iostream>
cout << "Message";
```

**Usage in Program:**
Displays initial loads, process migration steps, and final load distribution.

**2. Vector Library (<vector>)**

**Purpose:**
Used to store dynamic lists of nodes and processes.

**Syntax:**
```
#include <vector>
vector<int> v;
```

**Usage in Program:**
Stores loads of nodes and list of processes assigned to each node.

**3. Algorithm Library (<algorithm>)**

**Purpose:**
Provides functions for searching and comparing values.

**Syntax:**
```
#include <algorithm>
max_element(v.begin(), v.end());
```

**Usage in Program:**
Used to find the node with maximum and minimum load.

**2.2.1.2 Classes / Methods Used**

**1. struct Process**

**Purpose:**
Represents a process with ID and workload.

**Syntax:**
```
struct Process {
    int id;
    int load;
};
```

**Usage in Program:**
Stores process details for load transfer between nodes.

**2. vector**

**Purpose:**
Stores collection of elements dynamically.

**Syntax:**
```
vector<int> load;
vector<vector<Process>> nodes;
```

**Usage in Program:**

- load → stores load of each node
- nodes → stores processes assigned to nodes

**3. max_element()**
**Purpose:**
Finds the node with maximum load.
**Syntax:**
max_element(load.begin(), load.end());
**Usage in Program:**
Selects overloaded node.

**4. min_element()**
**Purpose:**
Finds the node with minimum load.
**Syntax:**
min_element(load.begin(), load.end());
**Usage in Program:**
Selects underloaded node.

**5. push_back()**
**Purpose:**
Adds an element to a vector.
**Syntax:**
v.push_back(element);
**Usage in Program:**
Adds a process to a node during migration.

**6. pop_back()**
**Purpose:**
Removes the last element from a vector.
**Syntax:**
v.pop_back();
**Usage in Program:**
Removes process from overloaded node.

**7. while loop**
**Purpose:**
Repeats execution until load is balanced.
**Syntax:**
while(condition) { }
**Usage in Program:**
Continuously balances load until threshold condition is satisfied.

## 3. Implementation

```
#include <iostream>
#include <vector>
#include <algorithm>
```

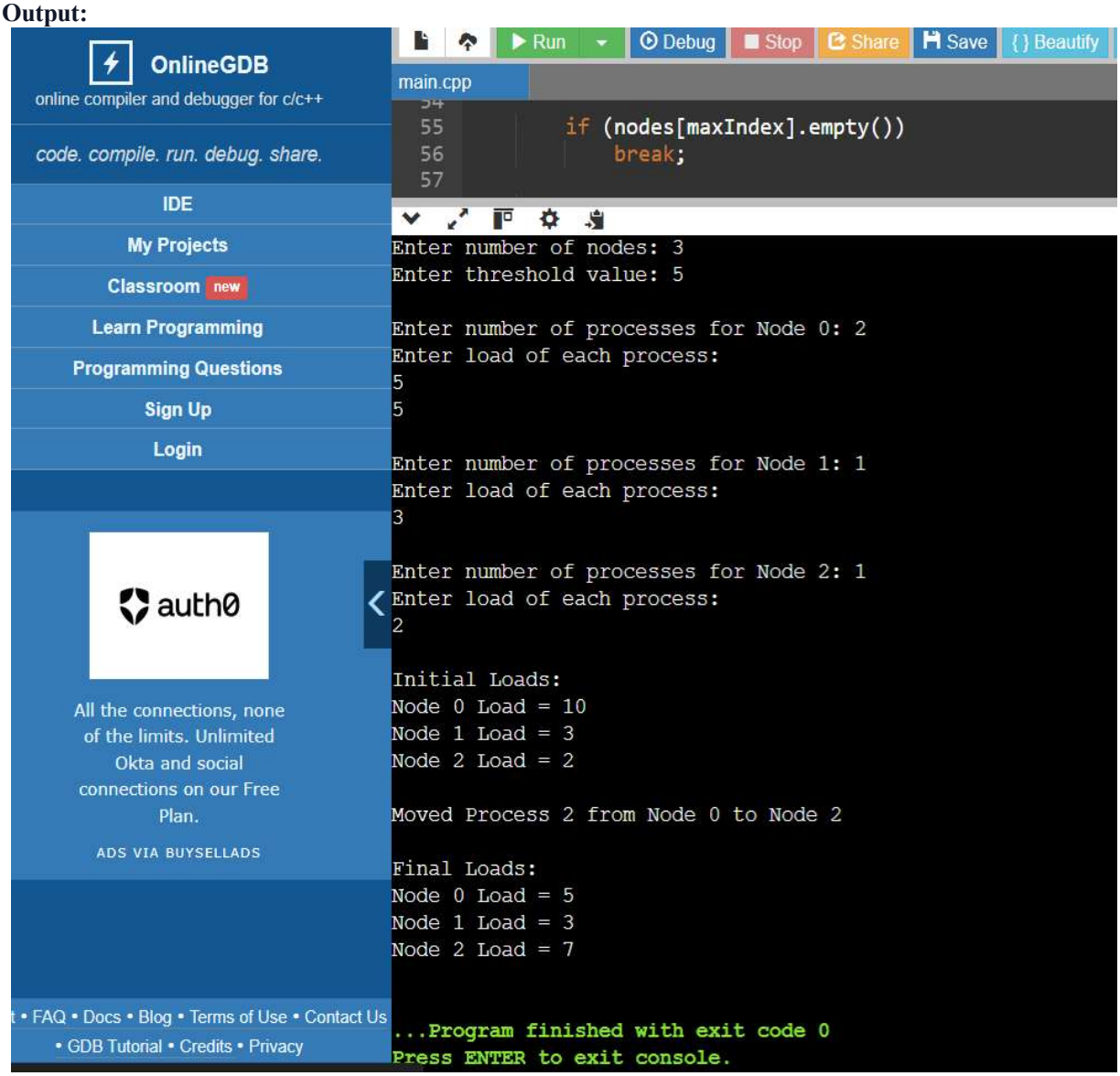
```
using namespace std;
// Structure for Process
struct Process {
    int id;
    int load;
};
int main() {
    int n, m, T;
    // Input number of nodes
    cout << "Enter number of nodes: ";
    cin >> n;
    // Input threshold
    cout << "Enter threshold value: ";
    cin >> T;
    vector<int> load(n);
    vector<vector<Process>> nodes(n);
    // Input processes for each node
    for (int i = 0; i < n; i++) {
        int p;
        cout << "\nEnter number of processes for Node " << i << ": ";
        cin >> p;
        cout << "Enter load of each process:\n";
        for (int j = 0; j < p; j++) {
            int l;
            cin >> l;
            nodes[i].push_back({j + 1, l});
            load[i] += l;
        }
    }
    // Display initial loads
    cout << "\nInitial Loads:\n";
    for (int i = 0; i < n; i++) {
        cout << "Node " << i << " Load = " << load[i] << endl;
    }
    // Load Balancing
    while (true) {
        int maxIndex = max_element(load.begin(), load.end()) -
load.begin();
        int minIndex = min_element(load.begin(), load.end()) -
load.begin();
        if (load[maxIndex] - load[minIndex] <= T)
            break;
        if (nodes[maxIndex].empty())
            break;
        // Select process from overloaded node
        Process p = nodes[maxIndex].back();
        nodes[maxIndex].pop_back();
```

```
        // Transfer process
        nodes[minIndex].push_back(p);

        // Update loads
        load[maxIndex] -= p.load;
        load[minIndex] += p.load;
        cout << "\nMoved Process " << p.id
            << " from Node " << maxIndex
            << " to Node " << minIndex << endl;
    }
    // Final loads
    cout << "\nFinal Loads:\n";
    for (int i = 0; i < n; i++) {
        cout << "Node " << i << " Load = " << load[i] << endl;
    }

    return 0;
}
```

**Command:** Run
**Platform:** Online C++ Compiler
**Output:**



**Conclusion:**

The implementation of load balancing in a distributed system demonstrates how workload can be efficiently distributed among multiple nodes to achieve optimal resource utilization and improved system performance. By identifying overloaded and underloaded nodes using threshold-based techniques, the system ensures that no single node becomes a bottleneck. The process migration strategy helps in dynamically redistributing tasks, thereby reducing response time and enhancing overall efficiency.

The program also highlights the importance of load estimation, process transfer policies, and decision-making mechanisms in maintaining system stability. Through practical implementation, it is observed that proper load balancing not only prevents system overload but also ensures fairness and better utilization of available resources.

Thus, load balancing plays a crucial role in distributed systems by improving scalability, reliability, and performance, making it an essential component in modern computing environments such as cloud computing and large-scale networked systems.

**SIGN AND REMARK**

**DATE**

| R1 (4 Marks) | R2 (4 Marks) | R3 (4 Marks) | R4 (3 Mark) | Total (15 Marks) | Signature |
|---|---|---|---|---|---|
| | | | | | |